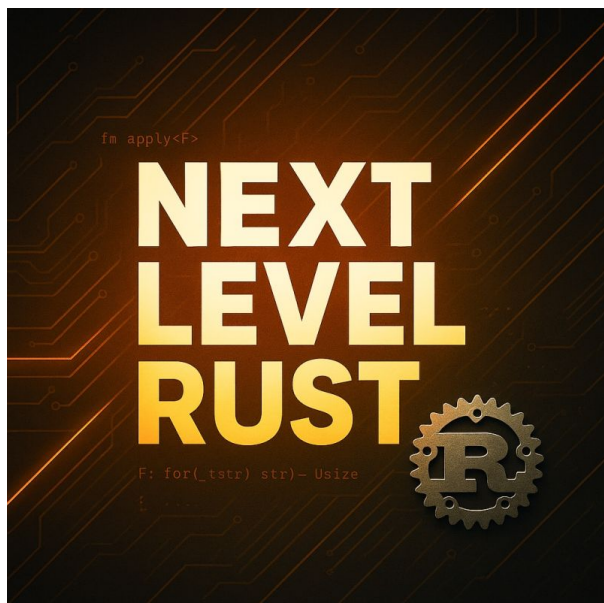




Copyrighted material

This is an integral part of course **Next level Rust** and cannot be used in any context without explicit permission from **CLEUTON SAMPAIO DE MELO JUNIOR** (a.k.a. "the Author"). You can't copy, change, publish, or share with others.

Advanced Cargo usage

Cargo Workspaces – Managing Multi-Crate Projects

In Rust, larger applications often grow beyond a single crate. You may want to split code into smaller, reusable parts: a **core library** with shared logic, a **CLI tool** for local use, and an **API library** for network services.

Cargo workspaces give you a clean way to organize all of them together.

Group crates under a single root `Cargo.toml`

At the top level, you create a root `Cargo.toml`. This file doesn't define a package itself. Instead, it defines a `[workspace]` section that ties multiple crates together.

Share build system, dependencies, and `target/` directory

All crates in a workspace share the same build system. That means:

- A single `target/` directory.
 - Dependencies are built once, then reused.
 - Builds are faster and consistent across crates.
-

Use **members** to list crates

Inside **[workspace]**, you declare the crates that belong. For example:

- **core** → shared business logic.
- **cli** → command-line app that uses **core**.
- **api** → network-facing library that also uses **core**.

Each crate has its own folder and **Cargo.toml**, but they live under one roof.

Enable **resolver = "2"**

This is critical when different crates in the workspace enable **different features** of the same dependency. With resolver v1, Cargo might unify features too aggressively, turning on more than you want. With resolver v2, Cargo unifies features only when necessary, respecting optional usage better.

Always add:

```
resolver = "2"
```

in your workspace root.

Example Project

Let's build **my_project** with three crates and a shared dependency on **serde**.

Root Cargo.toml

```
[workspace]
members = ["core", "api", "cli"]
resolver = "2"

[workspace.dependencies]
# Define serde ONCE for the whole workspace
serde = { version = "1.0", features = ["derive"] }
# If multiple crates use serde_json, centralize it too (optional)
serde_json = "1.0"
```

core/Cargo.toml

```
[package]
name = "core"
version = "0.1.0"
edition = "2021"
```

```
[dependencies]
# Inherit the shared serde config (with derive) from the workspace
serde = { workspace = true }
```

core/src/lib.rs

```
use serde::{Serialize, Deserialize};

#[derive(Serialize, Deserialize, Debug)]
pub struct Message {
    pub text: String,
}

pub fn greet(name: &str) -> Message {
    Message { text: format!("Hello, {}!", name) }
}
```

cli/Cargo.toml

```
[package]
name = "cli"
version = "0.1.0"
edition = "2021"

[dependencies]
core = { path = "../core" }
api = { path = "../api" }
# If the CLI also needs serde (e.g., config), inherit it:
serde = { workspace = true }
```

cli/src/main.rs

```
use core::greet;

fn main() {
    let msg = greet("from CLI");
    println!("{}", msg);
}
```

api/Cargo.toml

```
[package]
name = "api"
version = "0.1.0"
edition = "2021"

[dependencies]
core = { path = "../core" }
# Keep serde as a shared dep (even if this crate doesn't derive), per your
# requirement
serde = { workspace = true }
# Use serde_json for JSON work; DO NOT put features on `serde` for JSON
serde_json = { workspace = true }
```

api/src/lib.rs

```
use core::greet;
use serde_json;

pub fn api_greet() -> String {
    let msg = greet("from API");
    serde_json::to_string(&msg).unwrap()
}
```

In this workspace, the **root `Cargo.toml`** acts like a central configuration point, while each crate's **`Cargo.toml`** specifies its own local needs. When you put **`serde`** in the **`[workspace.dependencies]`** section at the root, you're declaring **one shared version and feature set** of **`serde`** for all crates.

Then, inside each member crate (**`core`**, **`api`**, **`cli`**), instead of declaring **`serde = "1.0"`** again (which could lead to mismatched versions or feature flags), we simply write:

```
serde = { workspace = true }
```

This tells Cargo: *"use the **`serde`** definition from the root workspace."* That way, all crates rely on the exact same version and features of **`serde`**.

Why this matters

- **Consistency:** Without this, one crate might use **`serde = "1.0", features=["derive"]`** while another pulls in plain **`serde = "1.0"`**. Cargo's feature resolver tries to unify them, but mistakes (like the invalid **`json`** feature) can cause conflicts.
- **Simplicity:** You only define the dependency once at the root. Updating **`serde`** or its features later requires changing a single place.
- **Shared dependencies pattern:** This is exactly what we call *shared dependencies* in Cargo workspaces — managing common crates centrally so that all members stay aligned.

So in this case:

- Root `Cargo.toml`: defines `serde` once with `features = ["derive"]`.
- `core`: inherits `serde` and actually uses the derive macros.
- `api` and `cli`: inherit `serde` too, even if they don't derive anything, so they remain consistent with the rest of the workspace.

This is a clean example of shared dependencies in Cargo workspaces.

We can check this using `cargo tree -e features -p serde` and see just one version of `serde` in the tree.

Why resolver = "2" matters

- The `core` crate only needs `serde` with `derive`.
- The `api` crate needs `serde` with both `derive` and `json`.
- With resolver v2, Cargo will unify these carefully:
 - `cli` will compile with only `derive`.
 - `api` will compile with `derive + json`.
 - Both reuse the same version of `serde`, but with just the features they need.

With resolver v1, features were sometimes applied **globally**, turning on extras in all crates even when not needed. Resolver v2 fixes this, making builds leaner and avoiding accidental feature bloat.

Recap

Cargo workspaces let you:

- Group multiple crates under one root.
- Share builds, dependencies, and target directory.
- List crates as `members`.
- Use `resolver = "2"` for correct feature unification.

The result: one project, many crates, consistent builds.